```
$ kubectl get deploy

NAME             READY   UP-TO-DATE   AVAILABLE   AGE
polar-keycloak   1/1     1            1           3m94s
polar-postgres   1/1     1            1           3m94s
polar-rabbitmq   1/1     1            1           3m94s
polar-redis      1/1     1            1           3m94s
polar-ui         1/1     1            1           3m94s
```

Let’s start by introducing ConfigMaps.

14.2.1 Configuring Spring Boot with ConfigMaps

In chapter 7, we used environment variables to pass hardcoded configuration to containers running in Kubernetes, but they lack maintainability and structure. ConfigMaps let you store configuration data in a structured, maintainable way. They can be version-controlled together with the rest of your Kubernetes deployment manifests and have the same nice properties of a dedicated configuration repository, including data persistence, auditing, and accountability.

A *ConfigMap* is “an API object used to store non-confidential data in key-value pairs. Pods can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a volume” (https://kubernetes.io/docs/concepts/configuration/configmap).

You can build a ConfigMap starting with a literal key/value pair string, with a file (for example, .properties or .yml), or even with a binary object. When working with Spring Boot applications, the most straightforward way to build a ConfigMap is to start with a property file.

Let’s look at an example. In the previous chapters, we configured Catalog Service via environment variables. For better maintainability and structure, let’s store some of those values in a ConfigMap.